📘 Section 8: Web Application & Production Deployment

📑 Table of Contents

1. Overview & Application Architecture

The user-facing deployment interface is implemented in `app.py`.

Built using **Streamlit**, the application provides a dark glassmorphism forensic dashboard where non-technical users, forensic analysts, and journalists can upload video files (`.mp4`, `.avi`, `.mov`) or static images (`.jpg`, `.png`) to receive real-time detection results, temporal timeline graphs, and 4-panel explainability diagnostics.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                            STREAMLIT WEB APPLICATION                         │
├──────────────────────────────────────────────────────────────────────────────┤
│ 1. Sidebar Control Panel: Threshold, Aggregation Method, Frame Sampling Slider │
│ 2. Video Player & Upload Zone: Supports MP4, AVI, MOV, JPG, PNG               │
│ 3. Calibrated Classification Card: Dynamic Red (Fake) / Green (Real) Card     │
│ 4. Temporal Anomaly Sequence Graph: Frame-by-Frame Matplotlib Confidence Timeline │
│ 5. Suspicious Faces Gallery: Top 4 Highest-Anomaly Face Crops                 │
│ 6. Explainability Inspector: On-Demand 4-Panel Diagnostics (RGB, SRM, FFT, Grad-CAM) │
└──────────────────────────────────────────────────────────────────────────────┘
```

---

2. Streamlit Server Execution Model

2.1 The Streamlit Reactive Rerun Loop

Unlike traditional web frameworks (Flask, FastAPI, Django) that use explicit route handlers, Streamlit executes the entire Python script from top to bottom whenever a user interacts with a widget (e.g., dragging a slider or uploading a file).

2.2 Singleton Model Caching with `@st.cache_resource`

Instantiating the `HybridDeepfakeDetector` and downloading/loading 200 MB of checkpoint weights on every slider interaction would introduce severe latency. In `app.py#L58-L70`, the model is loaded as a global singleton:

```
@st.cache_resource
def _cached_model_loader() → tuple[Any, Any, bool, float, float]:
    engine = load_prediction_engine()
    model, cropper, has_weights, threshold, temp = engine
    # Prime PyTorch CUDA / JIT kernels with a dummy forward pass
    try:
        dummy_tensor = torch.zeros(1, 3, 256, 256, device=DEVICE)
        with torch.inference_mode():
            model(dummy_tensor)
    except Exception as e:
        logger.debug("Model warm-up pass skipped: %s", e)
    return engine
```

- The `@st.cache_resource` decorator ensures the PyTorch model, weights, and YuNet cropper are initialized **only once** in server memory and shared across all user sessions.

2.3 CUDA / JIT Kernel Warm-Up Execution

The first time PyTorch executes a convolution or 2D FFT on a GPU, it invokes the CUDA driver to allocate memory and compile JIT kernels, causing a $2–3\,\text{second}$ cold-start delay. - The dummy forward pass (`model(dummy_tensor)`) inside `_cached_model_loader` warms up CUDA kernels during application startup, ensuring subsequent user uploads execute at full inference speed ($18\,\text{ms/crop}$).

---

3. File I/O & Windows Concurrency Safety

Implemented in `app.py#L43-L56` .

3.1 Temporary File Allocation (`tempfile.NamedTemporaryFile`)

When a user uploads a video file, Streamlit provides an in-memory `BytesIO` buffer. Because OpenCV's `cv2.VideoCapture` requires a physical filesystem path to stream compressed video codecs, the upload is written to a temporary file:

```
with tempfile.NamedTemporaryFile(delete=False, suffix=".mp4") as tfile:
    tfile.write(uploaded_file.read())
    temp_video_path = tfile.name
```

3.2 Windows File Lock Elimination (`safe_remove_file`)

On Windows operating systems, calling `os.unlink()` on a temporary file while OpenCV or background threads still hold a file handle raises a `PermissionError: [WinError 32] The process cannot access the file because it is being used by another process` .

`safe_remove_file` implements an exponential-backoff retry loop:

```
def safe_remove_file(file_path: str, max_retries: int = 3, delay: float = 0.5) → None:
    if not file_path or not os.path.exists(file_path):
        return
    for attempt in range(max_retries):
        try:
            os.unlink(file_path)
            return
        except PermissionError:
            if attempt < max_retries - 1:
                time.sleep(delay)
        except OSError:
            return
```

---

4. UI/UX Design System & Dark Glassmorphism

Implemented in `app.py#L82-L148`.

4.1 Color Palette & Typography (`Inter`)

The frontend is styled using raw CSS injected via `st.markdown(..., unsafe_allow_html=True)`: - **Background**: Deep Navy `#0b0f19` - **Surface Panels**: Slate Dark `#0f172a` - **Typography**: Modern Google Font `'Inter', -apple-system, BlinkMacSystemFont, sans-serif` - **Accent Primary**: Deep Blue `rgba(37, 99, 235, 0.15)` - **Accent Secondary**: Purple `rgba(147, 51, 234, 0.15)`

4.2 Glassmorphism Container Styling

Containers use CSS backdrop filters for a translucent frosted-glass aesthetic:

```
.card-workflow {
    background: rgba(30, 41, 59, 0.5);
    border: 1px solid rgba(255, 255, 255, 0.08);
    border-radius: 12px;
    padding: 18px;
    backdrop-filter: blur(12px);
}
```

4.3 Dynamic Verdict Cards (Fake vs. Real)

Depending on whether the video prediction score exceeds the threshold ($p \geq \theta$), the verdict card updates dynamically: - **Fake Detection Card**: `css .result-card-fake { background: rgba(220, 38, 38, 0.12); border: 2px solid #ef4444; box-shadow: 0 8px 24px rgba(239, 68, 68, 0.2); }` - **Real Detection Card**: `css .result-card-real { background: rgba(34, 197, 94, 0.12); border: 2px solid #22c55e; box-shadow: 0 8px 24px rgba(34, 197, 94, 0.2); }`

4.4 Mobile Responsive Media Breakpoints

```
@media (max-width: 640px) {
    [data-testid="column"] {
        width: 100% !important;
        flex: 1 1 100% !important;
        min-width: 100% !important;
    }
}
```

Ensures columns collapse gracefully to single-column vertical stacks on mobile devices.

---

5. Interactive Sidebar & Forensic Control Panel

Implemented in `app.py#L150-L210`.

The Streamlit sidebar provides interactive forensic controls:

5.1 Decision Threshold Slider ($\theta \in [0.00, 1.00]$)

Allows analysts to adjust the decision boundary: - **Default**: $\theta = 0.01$ (calibrated operating point for 99.8% Fake Recall). - **Conservative Setting ($\theta = 0.50$)**: Balanced operating point. - **Strict Setting ($\theta = 0.80$)**: High-precision mode requiring overwhelming synthetic evidence before flagging.

5.2 Temporal Aggregation Policy Selector

Allows toggling between multi-frame pooling algorithms: - `Softmax-Weighted (tau=0.10)` (Default recommended) - `Top-k Anomaly Average` - `Exponential Moving Average (EMA)` - `Arithmetic Mean`

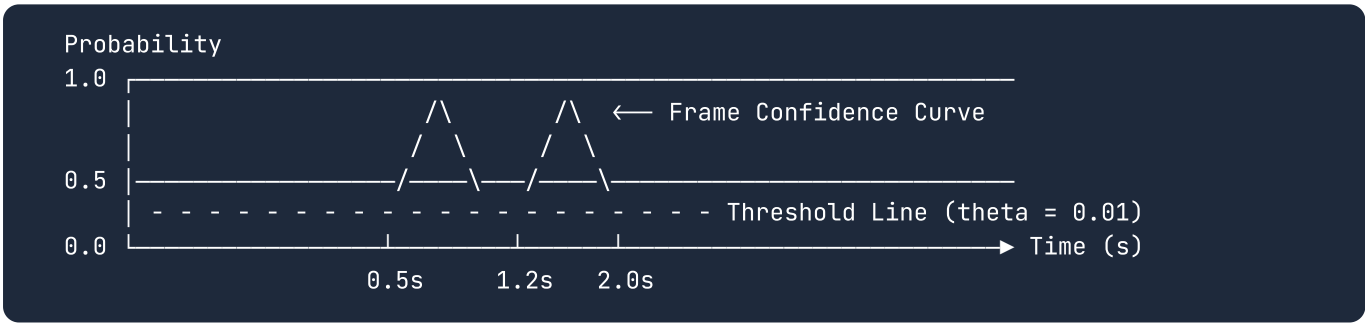
5.3 Frame Sampling Density Slider ($5 \dots 50$)

Controls how many uniformly spaced frames are extracted from the uploaded video (default: 16 frames).

---

6. Temporal Anomaly Sequence Visualization

Implemented in `render_temporal_anomaly_timeline`.



6.1 Matplotlib Dark Canvas Architecture

- Figure Face Color: `#0b0f19`
- Axes Face Color: `#0f172a`
- Grid Lines: `#1e293b` dotted grid

6.2 Anomaly Area Shading (Red vs. Green)

Using Matplotlib's `fill_between`: - Regions where $p(t) \geq \theta$ are shaded in **translucent red** (`#ef4444`, $\alpha = 0.22$). - Regions where $p(t) < \theta$ are shaded in **translucent green** (`#22c55e`, $\alpha = 0.12$).

6.3 Timestamped Scatter Points & Threshold Intersections

Scatter points mark individual frame evaluations: - Red markers indicate anomalous frames. - Green markers indicate authentic frames. - The red dashed horizontal line marks the active decision threshold.

---

7. Top-4 Suspicious Face Gallery & Diagnostic Inspector

Implemented in `app.py#L380-L460`.

7.1 Ranked Anomaly Extraction

The inference engine sorts all extracted faces in descending order of fake probability:

```
zipped_data = list(zip(all_faces, all_probs))
zipped_data.sort(key=lambda x: x[1], reverse=True)
top_4 = zipped_data[:4]
```

The 4 highest-scoring crops are displayed in a 4-column gallery with their individual frame confidence percentages.

7.2 Interactive 4-Panel Forensic Diagnostics Tab

Clicking on any suspicious face generates the full 4-panel explainability visualization: - Panel (a): RGB Crop - Panel (b): SRM Noise Residual Map (Viridis) - Panel (c): 2D FFT Centered Log-Magnitude Spectrum (Magma) - Panel (d): ConvNeXt Grad-CAM Attention Overlay

---

8. Production Serving & Deployment Options

8.1 Hugging Face Spaces Cloud Deployment

Configured via YAML frontmatter in `README.md`:

```
---
title: Deepfake Detection Engine
colorFrom: blue
colorTo: purple
sdk: streamlit
sdk_version: 1.32.0
app_file: app.py
pinned: false
license: mit
---
```

- **Live Space URL**: https://huggingface.co/spaces/yyouretoast/deepfake-detector

8.2 Docker Containerization (`Dockerfile`)

The repository includes a standalone `Dockerfile` for containerized deployment:

```
FROM python:3.10-slim
WORKDIR /app
RUN apt-get update && apt-get install -y libgl1 libglib2.0-0 && rm -rf /var/lib/apt/lists/*
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

8.3 ONNX Runtime Export Pipeline (`export_onnx.py`)

Implemented in `scripts/export_onnx.py` : - Exports the PyTorch `HybridDeepfakeDetector` to ONNX format (`models/deepfake_detector.onnx`) for cross-platform deployment via C++, Rust, or TensorRT.

---

9. Code Walkthrough & Reference

Component / Function	File Reference	Primary Responsibility
`render_ui`	`app.py#L73-L597`	Master Streamlit layout, reactive event loop, upload handler, and UI rendering.
`_cached_model_loader`	`app.py#L58-L70`	Singleton model loader with CUDA kernel warmup pass.
`safe_remove_file`	`app.py#L43-L56`	Retry handler for Windows file locks.
`render_temporal_anomaly_timeline`	`src/utils/visualization.py#L10-L100`	Dark-themed Matplotlib temporal anomaly graph generator.
`Dockerfile`	`Dockerfile`	Production Docker container specification.
`export_onnx.py`	`scripts/export_onnx.py`	ONNX model exporter for high-performance edge deployment.

This document serves as the permanent reference for Section 8 (Web Application & Production Deployment) of the Deepfake Detection Engine.